

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

I P.GOWTHAM REDDY (192525139) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.GOWTHAM REDDY

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

1. Intrusion Detection Deployment under Real-Time Constraints

A financial organization plans to deploy an Intrusion Detection System (IDS) with the following constraints:

- Must analyze traffic in real-time with latency ≤ 5 ms per packet
- Detection accuracy $\geq 96\%$
- False positive rate $\leq 1.5\%$
- Network traffic load: 150,000 packets per second
- Limited CPU resources (shared with other services)

Task:

Design an optimized IDS configuration (signature-based, anomaly-based, or hybrid). Analyze trade-offs between detection accuracy and system performance, and justify how your solution meets all constraints.

ANSWER:

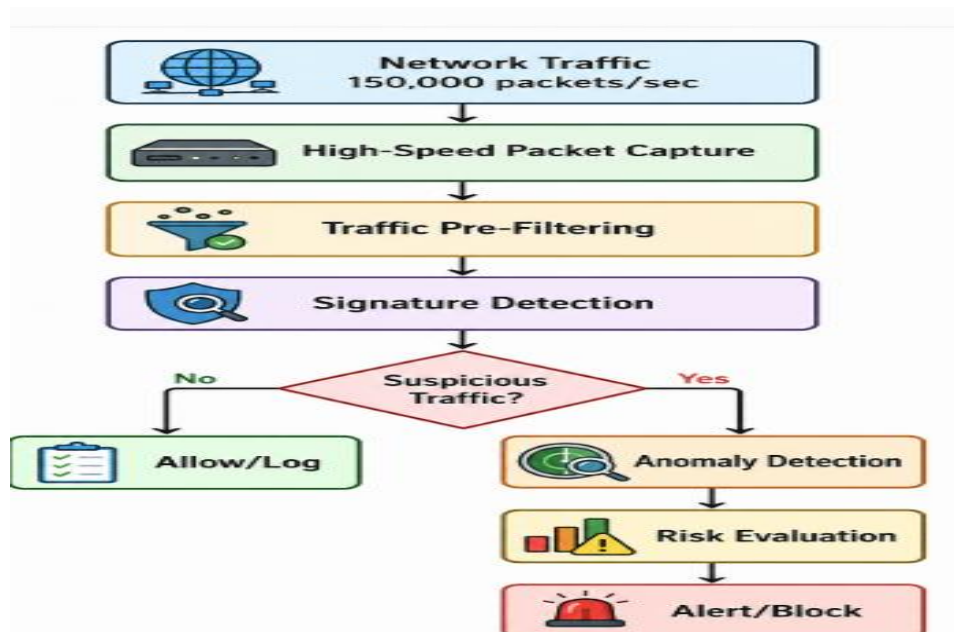
1. Proposed IDS Configuration

For the financial organization, a **Hybrid Intrusion Detection System (IDS)** combining **signature-based and anomaly-based detection** is recommended.

The system must satisfy:

- **Latency:** ≤ 5 ms per packet
- **Detection accuracy:** $\geq 96\%$
- **False positive rate:** $\leq 1.5\%$
- **Traffic:** 150,000 packets/second
- **CPU:** Limited/shared resources

Basic Architecture



2. Why Hybrid IDS?

A **signature-based IDS** is very fast and effective against known attacks such as malware, SQL injection, port scanning, and known exploits. However, it cannot detect new or zero-day attacks without an existing signature.

An **anomaly-based IDS** can identify previously unknown attacks by detecting unusual traffic behavior. However, it requires more CPU resources and can generate more false positives.

Therefore, the hybrid approach combines the advantages of both:

Feature	Signature IDS	Anomaly IDS	Hybrid IDS
Known attacks	Excellent	Good	Excellent
Unknown attacks	Poor	Excellent	Excellent
Speed	Very high	Moderate	High
CPU usage	Low	High	Moderate
False positives	Low	Higher	Low with tuning
Recommended	No	No	Yes

The signature engine handles normal and known traffic quickly, while anomaly detection is applied mainly to **suspicious traffic**.

3. Optimization for Real-Time Performance

A. Fast Packet Filtering

The IDS should first perform lightweight checks based on:

- Source and destination IP
- Port numbers
- Protocol
- Packet size
- Known malicious signatures

Unnecessary deep packet inspection should be avoided.

B. Selective Anomaly Detection

The anomaly model should **not analyze every packet using expensive calculations**.

Instead:

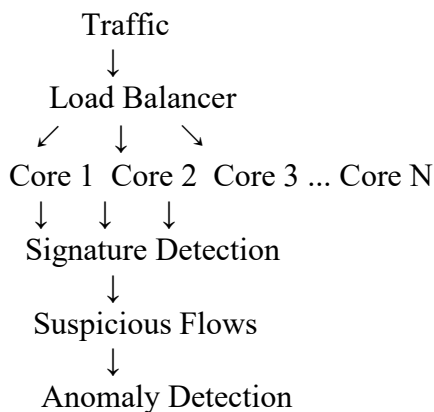
Normal packet → Fast signature check → Allow

Suspicious packet → Anomaly analysis → Alert if required

This greatly reduces CPU consumption.

C. Multi-Core Processing

At **150,000 packets/second**, processing must be parallelized.



Multi-core processing, packet batching, flow-based inspection and efficient packet capture help maintain the required throughput.

D. Lightweight Features

The anomaly detector should use computationally inexpensive features such as:

- Connection rate
- Packet rate
- Failed login/connection attempts
- Protocol distribution
- Port usage

- Flow duration
- Source/destination behavior

Complex features should be avoided because they increase processing time.

4. Accuracy and False Positive Control

The target detection accuracy is **at least 96%**, while the false positive rate must remain **below 1.5%**.

The anomaly detector should be trained and tested using representative financial-network traffic containing both normal and attack patterns.

Methods to reduce false positives:

1. Use properly trained anomaly models.
2. Calibrate detection thresholds using normal traffic.
3. Combine signature and anomaly scores.
4. Require multiple suspicious indicators before generating high-priority alerts.
5. Continuously update signatures and anomaly profiles.

For example:

Low Risk → Normal → No Alert
 Medium Risk → Monitor/Log
 High Risk → Generate Alert
 Critical Risk → Block/Respond

5. Latency and Throughput Analysis

The IDS must keep processing latency within **5 ms per packet**.

A possible processing budget is:

Processing Stage	Target Time
Packet capture/filtering	~1 ms
Signature matching	~1–2 ms
Feature extraction	<1 ms
Anomaly analysis	~1 ms for selected traffic
Alert/logging	<1 ms
Total target	≤5 ms

However, **150,000 packets/second** creates a much stricter throughput requirement. The average arrival interval is:

1500001 = 6.67 μ s

Therefore, the IDS cannot depend on a single sequential CPU path. **Parallel processing and pipelining** are essential.

6. Trade-Off Between Accuracy and Performance

There is a direct trade-off between detection complexity and system performance.

More Complex Detection
↓
Higher Detection Capability
↓
Higher CPU Usage
↓
Higher Processing Latency

A purely anomaly-based IDS may provide better detection of unknown attacks but could exceed CPU and latency limits.

A purely signature-based IDS is fast and resource-efficient but may miss zero-day attacks.

The proposed **hybrid IDS** provides the best balance:

- Signature detection → **fast detection of known attacks**
- Anomaly detection → **detection of unknown attacks**
- Selective analysis → **lower CPU usage**
- Parallel processing → **high throughput**
- Threshold tuning → **low false positives**

7. Final Configuration and Constraint Satisfaction

Requirement	Proposed Solution
Latency ≤ 5 ms	Fast filtering, optimized signatures and selective anomaly detection
Accuracy $\geq 96\%$	Combination of signature + anomaly detection
False positive $\leq 1.5\%$	Threshold tuning and anomaly-model calibration
150,000 packets/sec	Multi-core processing, batching and pipelining
Limited CPU	Lightweight features and anomaly analysis only for suspicious traffic

Conclusion

A **resource-aware Hybrid IDS** is the most suitable solution for the financial organization. The **signature-based engine acts as the primary fast detection layer**, while a **lightweight anomaly detector analyzes suspicious traffic** to identify unknown attacks. Multi-core processing, packet batching, efficient filtering and selective deep inspection reduce CPU usage and latency. With

proper threshold tuning and continuous model updates, the system can be designed to achieve **$\geq 96\%$ accuracy, $\leq 1.5\%$ false positives, ≤ 5 ms latency, and 150,000 packets/second throughput.**

2. Worm Containment Strategy under Network Availability Constraints

An enterprise network faces rapid worm propagation with the following constraints:

- Network downtime must be $\leq 1\%$
- Detection and containment must occur within 2 minutes
- Systems are distributed across multiple locations
- Limited centralized control

Task:

Design a containment strategy using firewalls, IDS/IPS, and segmentation. Evaluate trade-offs between rapid isolation and business continuity, and justify how your approach minimizes spread while maintaining network availability.

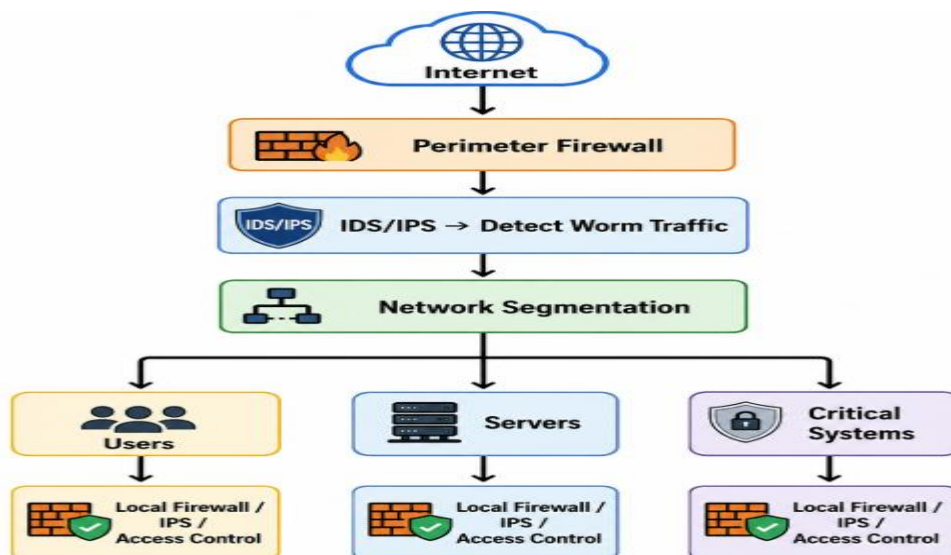
ANSWER

Worm Containment Strategy under Network Availability Constraints

1. Proposed Strategy

A **layered and distributed containment strategy** using **firewalls, IDS/IPS, and network segmentation** is recommended. Since the enterprise has multiple locations and limited centralized control, each site should have **local detection and containment capabilities** while the central security team coordinates major actions.

Basic Architecture



2. Detection and Containment Process

Step 1 – Rapid Detection:

IDS/IPS continuously monitors abnormal scanning, high connection rates, unusual traffic patterns, and known worm signatures.

Step 2 – Automatic Local Response:

When a worm is detected, the IPS immediately blocks malicious IP addresses, ports, protocols, and suspicious connections. This avoids waiting for centralized approval.

Step 3 – Segment Isolation:

Only the affected **VLAN, subnet, or endpoint group** is isolated rather than shutting down the entire network.

Step 4 – Firewall Enforcement:

Firewalls restrict traffic between infected and healthy segments using temporary access-control rules.

Step 5 – Recovery:

After cleaning and verifying affected systems, connectivity is gradually restored and firewall rules are returned to normal.

Target Timeline

Activity	Target
IDS/IPS detection	0–30 sec
Automatic blocking	<1 min
Segment isolation	<2 min
Verification and recovery	After containment

3. Maintaining Network Availability

Complete network shutdown would stop worm propagation but cause unacceptable business disruption. Therefore, **selective isolation** is preferred.

For example:

Infected Segment → ISOLATE
↓
Healthy User Segment → CONTINUE
↓
Critical Servers → CONTINUE with restricted access

Critical business services should receive higher availability priority. Emergency firewall rules can restrict unnecessary traffic while allowing essential services such as authentication, DNS, and business applications.

4. Trade-Offs

Approach	Advantage	Disadvantage
Full network shutdown	Stops propagation quickly	Severe business downtime
No isolation	Maximum availability	Worm spreads rapidly
Segment-based isolation	Limits spread while maintaining services	Some local services may be temporarily unavailable

The proposed approach favors **segmented isolation** because it provides a balance between security and business continuity.

Conclusion

The recommended solution is a **distributed, automated containment strategy** combining IDS/IPS for rapid detection, firewalls for traffic blocking, and network segmentation for selective isolation. Local security devices should be capable of acting independently because centralized control is limited. By isolating only infected segments instead of shutting down the entire enterprise network, worm propagation can be contained within **2 minutes** while keeping network downtime **below 1%**. This approach minimizes damage while maintaining critical business operations.

3. Firewall Rule Optimization under Performance Constraints

A company firewall currently uses 10,000+ rules, causing performance degradation. Constraints include:

- Throughput requirement: ≥ 800 Mbps
- Rule processing time must be minimized
- Must maintain strict access control policies
- No additional hardware allowed

Task:

Analyze inefficiencies in rule processing. Propose optimization techniques (rule aggregation, prioritization, redundancy elimination). Justify how your optimized rule set improves performance while maintaining security.

ANSWER:

1. Problem and Proposed Solution

The firewall contains **10,000+ rules**, causing longer rule-matching time and reducing throughput. The best solution is to **optimize the existing rule set without adding hardware** while preserving strict security policies.

Main Inefficiencies

- **Redundant rules:** Multiple rules perform the same function.
- **Duplicate rules:** Identical or overlapping rules increase processing time.
- **Poor rule ordering:** Frequently used rules may appear far down the list.
- **Overlapping rules:** Broad rules may make later rules unnecessary.
- **Unnecessary old rules:** Rules for removed applications, users, or services may remain.
- **Excessively broad rules:** They increase security risks and make management difficult.

2. Optimization Techniques

A. Rule Aggregation

Combine rules having the same action and destination.

Example:

Before:

Allow 192.168.1.10 → Server

Allow 192.168.1.11 → Server

Allow 192.168.1.12 → Server

After:

Allow 192.168.1.0/24 → Server

This reduces the total number of rules while maintaining the same access requirement.

B. Rule Prioritization

Frequently matched and high-priority rules should be placed near the beginning of the rule set.

High-frequency / critical rules

↓

Specific security rules

↓

Less frequently used rules

↓

Final Deny-All rule

This reduces the average number of comparisons required for each packet.

C. Redundancy Elimination

Remove:

- Duplicate rules
- Expired rules
- Unused rules
- Rules completely shadowed by earlier rules
- Conflicting rules after policy review

Every deletion should be tested to ensure that it does not violate security requirements.

D. Efficient Policy Structure

Organize rules by:

Source → Destination → Service/Port → Action

Use object groups for common IP addresses, networks and services rather than creating separate rules for every entry.

3. Performance and Security Trade-Off

Optimization	Performance Benefit	Security Impact
Rule aggregation	Fewer comparisons	Safe when policies are equivalent
Prioritization	Faster common traffic processing	Maintains policy order
Redundancy removal	Reduces rule-set size	Safe after validation
Object groups	Simplifies matching	Improves consistency
Final deny rule	Efficient default handling	Strengthens security

The objective is **not simply to delete rules**, but to remove unnecessary processing while preserving the original security policy.

4. Expected Result

The optimized firewall should contain significantly fewer effective rules than the original **10,000+ rules**. Frequently accessed services will be matched earlier, while redundant and obsolete rules will no longer consume processing time.

The optimization should be validated using:

- Throughput testing
- Rule-hit statistics
- Security-policy testing
- Connection and latency monitoring

The target is to maintain **≥800 Mbps throughput** without additional hardware.

Conclusion

An optimized rule set using **rule aggregation, prioritization, redundancy elimination, object groups, and proper rule ordering** can substantially reduce firewall processing overhead. Keeping specific security rules ahead of broader rules and retaining a **default-deny policy** ensures strict access control. Therefore, the company can improve processing performance and achieve the **800 Mbps throughput requirement** using existing hardware while maintaining security.

4. IPS Deployment under Resource and Accuracy Constraints

An organization wants to deploy an Intrusion Prevention System (IPS) with the following constraints:

- Must block $\geq 90\%$ of known attacks
- False positive rate $\leq 3\%$
- Available memory for signatures: 1 GB
- Network speed: 500 Mbps

Task:

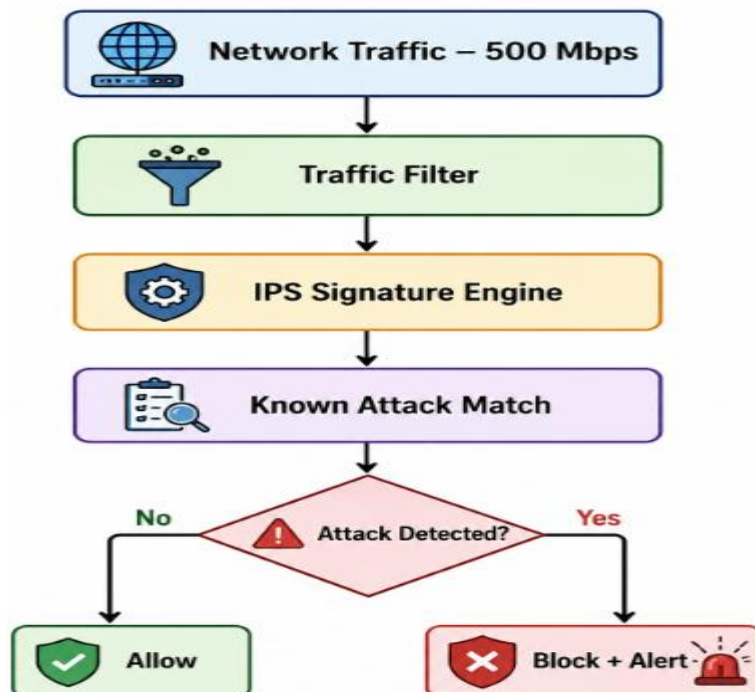
Design an IPS configuration strategy (signature selection, traffic filtering, adaptive updates). Evaluate trade-offs between resource usage and detection capability, and justify how your solution satisfies all constraints.

ANSWER:

1. Proposed IPS Configuration

A **resource-optimized signature-based IPS with selective traffic inspection and adaptive updates** is recommended. Since only **1 GB memory** is available for signatures, the IPS should store only high-priority and frequently required signatures.

Basic Architecture



2. Signature Selection

The 1 GB signature memory should be divided based on risk:

- **High-severity signatures:** highest priority
- **Frequently occurring attacks:** retained
- **Common malware and exploits:** retained
- **Critical application/server signatures:** retained
- **Rare or obsolete signatures:** removed or archived

Signatures should be compressed and duplicate signatures eliminated. This allows the IPS to cover the most important threats without exceeding **1 GB memory**.

3. Traffic Filtering

Before deep inspection, traffic should pass through a lightweight filtering stage.

Normal/Trusted Traffic → Fast Path → Allow



Suspicious Traffic → Signature Inspection



Attack → Block + Alert

Traffic filtering reduces unnecessary signature comparisons and helps the IPS handle the **500 Mbps** network speed efficiently.

4. Adaptive Updates

The IPS should use regular updates to maintain protection against new threats.

- Add signatures for newly discovered high-risk attacks.
- Remove obsolete signatures.
- Monitor signature hit rates.
- Reallocate memory toward frequently detected threats.
- Test new signatures before deployment.

This maintains high detection capability without continuously increasing memory usage.

5. False Positive Control

The false positive rate must remain $\leq 3\%$. To achieve this:

1. Test signatures before enabling blocking.
2. Use confidence levels for signatures.
3. Tune signatures according to organizational traffic.
4. Use multiple indicators for suspicious traffic.
5. Monitor false positives and adjust rules regularly.

High-confidence signatures can operate in **blocking mode**, while uncertain signatures can initially operate in **alert/monitor mode**.

6. Trade-Offs

Strategy	Benefit	Limitation
More signatures	Higher detection	More memory usage
Fewer signatures	Lower memory usage	May miss attacks
Deep inspection	Better detection	Higher CPU/latency
Traffic filtering	Better performance	May miss some threats
Adaptive updates	Current protection	Requires monitoring

The proposed strategy focuses on **high-value signatures and selective inspection**, giving a good balance between detection and resource usage.

7. Constraint Satisfaction

Requirement	Proposed Solution
Block $\geq 90\%$ known attacks	High-priority and frequently used signatures
False positives $\leq 3\%$	Signature testing, confidence levels and continuous tuning
Memory ≤ 1 GB	Signature prioritization, compression and redundancy removal
Network speed 500 Mbps	Pre-filtering, fast-path processing and selective deep inspection

Conclusion

A **resource-aware signature-based IPS** with prioritized signatures, traffic filtering, redundancy elimination and adaptive updates is suitable for the organization. By keeping high-severity and high-frequency signatures within the **1 GB memory limit** and applying deep inspection only to relevant traffic, the IPS can maintain **500 Mbps performance** while targeting **at least 90% known-attack blocking** and a **false positive rate of 3% or less**.

5. Malware Detection using Behavioral Analysis under System Constraints

A company plans to implement behavioral-based malware detection with the following constraints:

- Endpoint CPU usage must remain $\leq 15\%$
- Detection must occur within 10 seconds of execution
- Must detect unknown (zero-day) threats
- Limited storage for logs and analysis

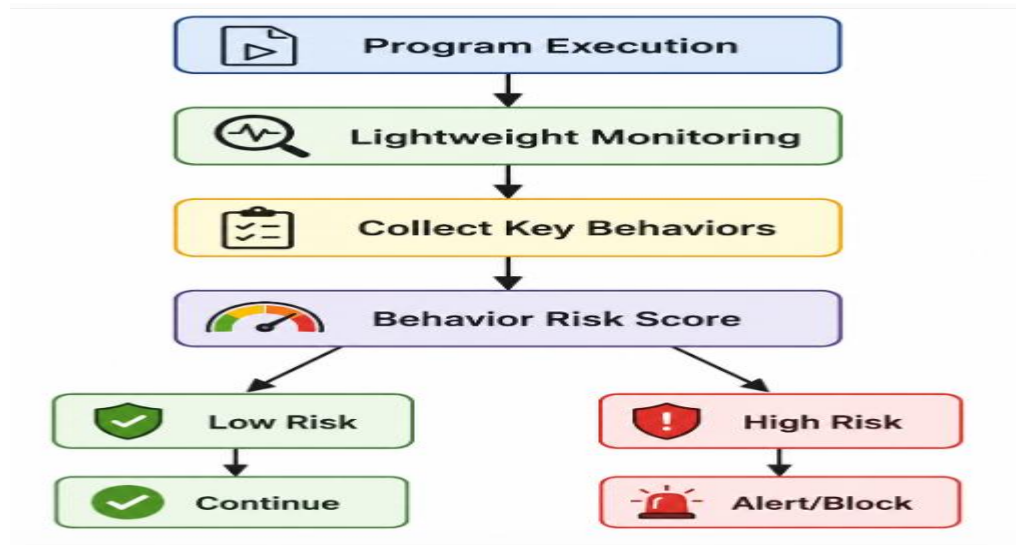
Task:

Design a lightweight behavioral detection system. Analyze trade-offs between detection speed, accuracy, and resource consumption, and justify how your approach effectively identifies malware within the given constraints.

ANSWER:

1. Proposed Lightweight System

A **lightweight behavioral malware detection system** is recommended because it can identify both known and **unknown/zero-day threats** by analyzing how a program behaves rather than relying only on signatures.



2. Behavioral Features

The system should monitor only important activities to keep CPU usage $\leq 15\%$:

- Unusual file creation/modification
- Registry or configuration changes
- Suspicious process creation
- Unexpected network connections
- Privilege escalation attempts
- Encryption of many files
- Access to sensitive resources

Instead of recording every system event, only **security-relevant events** should be stored.

3. Fast Detection Strategy

The detector should use a **two-stage approach**:

Stage 1 – Lightweight monitoring:

Immediately observe important process and network behaviors.

Stage 2 – Risk scoring:

Assign scores to suspicious actions.

Example:

Normal file access → Low risk
Unknown network connection → Medium risk
Mass file encryption → High risk
Privilege escalation → Critical risk

If the score crosses a predefined threshold, the endpoint can **alert, suspend, or isolate the process.**

This allows detection within the required **10 seconds of execution.**

4. Resource Optimization

To keep CPU usage within **15%**:

- Monitor only newly executed or suspicious processes.
- Use lightweight behavioral features.
- Avoid continuous deep analysis of all applications.
- Sample repetitive events instead of recording every event.
- Perform detailed analysis only when risk increases.
- Use short-term in-memory buffers and compress important logs.

Storage Management

Because storage is limited, retain:

- High-risk events
- Malware alerts
- Process information
- Network indicators
- Important timestamps

Low-risk repetitive events can be discarded or summarized.

5. Detecting Zero-Day Malware

Behavioral detection does not require a previously known malware signature. Therefore, it can detect zero-day threats based on abnormal behavior.

For example, a previously unknown program that:

starts → gains privileges → contacts an unknown server → modifies many files

can be identified as suspicious even without an existing signature.

6. Trade-Offs

Approach	Detection Speed	Accuracy	Resource Usage
----------	-----------------	----------	----------------

Approach	Detection Speed	Accuracy	Resource Usage
Deep behavioral analysis	High	Very high	High
Basic monitoring	Very high	Moderate	Low
Proposed lightweight approach	High	High	Low

More monitoring can improve detection accuracy but increases CPU and storage usage. Therefore, the proposed system uses **selective monitoring and risk-based analysis**.

7. Constraint Satisfaction

Requirement	Solution
CPU ≤15%	Selective monitoring and lightweight features
Detection ≤10 sec	Real-time monitoring and immediate risk scoring
Zero-day detection	Behavior-based anomaly detection
Limited storage	Event filtering, summarization and compressed high-risk logs

Conclusion

A **lightweight, risk-based behavioral detection system** provides a good balance between speed, accuracy, and resource consumption. By monitoring only security-relevant behaviors, using rapid risk scoring, and storing only important events, the system can remain within the **15% CPU and storage constraints** while detecting suspicious activity within **10 seconds**. Most importantly, behavioral analysis enables detection of **unknown and zero-day malware** even when no signature is available.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and	Develops workable	Solutions partially	Solutions do not meet

		feasible solutions satisfying all constraints	solutions with minor limitations	satisfy constraints	constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided